SIMATS ENGINEERING

ASSESSMENT TOOL 1 - Concept Mapping

COURSE NAME: Data Structures **COURSE CODE: CSA0308**

Register Number	**192525071**
Name	**Kanithi Ramya**

COURSE OUTCOME COVERED

CO1: *Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity.* (BL2, BL3)

Assessment Tool Weightage: 35%

Questions:2	**EACH QUESTIONS CARRIES:** 10 MARKS	**Total Marks:**20	**Duration:** ONE HOUR

Q1 A smart healthcare system processes large volumes of patient data such as temperature readings, medical history, and diagnostic details. Construct a concept map linking the progression from Data → Data Object → Data Structure → Abstract Data Type (ADT) → Implementation in C. Identify relevant sub-concepts such as structures, pointers, and operations, and establish meaningful relationships among them. Organize the concept map hierarchically and justify how each level contributes to efficient data organization, processing, and improved system performance.

Q2 A smart city traffic management system handles data related to vehicle movement, road networks, and signal control. Construct a concept map connecting Primitive Data Structures → Non-Primitive Data Structures → Linear → Non-Linear Structures. Include examples such as arrays, linked lists, trees, and graphs, and illustrate their applications in traffic management. Explain the relationships between these structures and justify their classification based on functionality and efficiency.

Code of Conduct:

*I **(Kanithi Ramya / 192525071)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

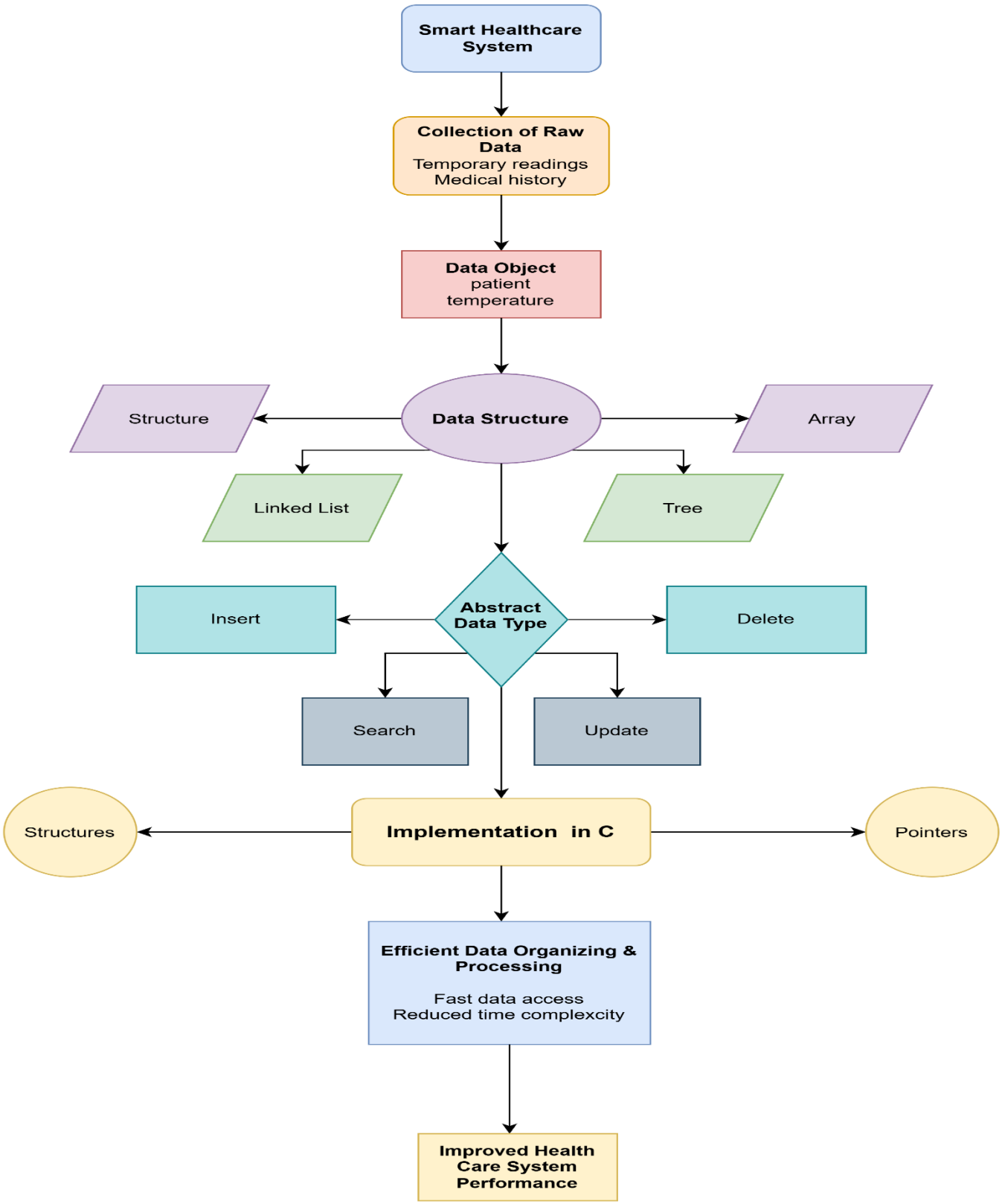
Signature of the student: K.Ramya

RUBRICS FOR CALCULATION:

Concept Mapping					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Answers

1) Concept Map



Smart Healthcare System:

Explanation of the Concept Map

1. Data

Data refers to raw facts collected by the smart healthcare system. Examples include:

- Patient ID
- Temperature
- Blood pressure
- Medical history
- Diagnostic details

For example:

Temperature = 38.5°C

This raw information is the starting point of the data-processing system.

---

2. Data Object

A **data object** represents a meaningful entity created from the collected data.

For example:

Patient Object

It can contain:

Patient ID → P101

Temperature → 38.5°C

Diagnosis → Fever

Medical History → Diabetes

Thus, data objects convert individual pieces of raw data into meaningful entities.

---

3. Data Structure

A **data structure** organizes and stores data so that it can be accessed and processed efficiently.

Important data structures include:

- **Array** – stores multiple similar values.
- **Structure** – groups different types of information.
- **Linked List** – stores dynamically changing patient records.
- **Pointers** – store addresses and help access or connect data.

For example, a patient record can be represented using a C structure:

```
struct Patient
{
   int id;
   float temperature;
   char diagnosis[30];
};
```

Here, the structure groups different types of patient information into one record.

---

4. Abstract Data Type (ADT)

An **Abstract Data Type** defines **what data can be stored and what operations can be performed**, without specifying the actual implementation.

For a healthcare patient-record ADT, the operations may include:

INSERT
DELETE
SEARCH
UPDATE
DISPLAY
SORT

For example:

Patient ADT

→ Insert a new patient
→ Search for a patient
→ Update patient information
→ Delete a patient record
→ Display patient details

The ADT focuses on **what operations are required**, while the implementation decides **how they are performed**.

---

5. Implementation in C

The ADT can be implemented in the C programming language using:

- struct
- Arrays
- Pointers
- Functions
- Dynamic memory allocation

Example:

```
#include <stdio.h>

struct Patient
{
   int id;
   float temperature;
   char diagnosis[30];
};

int main()
{
   struct Patient p;
```

```
    printf("Enter Patient ID: ");
    scanf("%d", &p.id);

    printf("Enter Temperature: ");
    scanf("%f", &p.temperature);

    printf("Enter Diagnosis: ");
    scanf("%s", p.diagnosis);

    printf("\nPatient Details\n");
    printf("ID: %d\n", p.id);
    printf("Temperature: %.2f\n", p.temperature);
    printf("Diagnosis: %s\n", p.diagnosis);

    return 0;
}
```

This shows how a patient data object can be represented and processed in C.

Role of Pointers

Pointers store the memory address of data.

They are useful because they:

- Provide direct access to data.
- Allow structures to be connected.
- Support linked lists.
- Help in dynamic memory allocation.
- Reduce unnecessary copying of large data.

For example:

struct Patient *ptr;

The pointer can point to a patient structure and provide efficient access to its information.

Relationship Between the Concepts

The overall progression can be summarized as:

DATA

↓

Raw patient information

↓

DATA OBJECT

↓

Meaningful patient entity

↓

DATA STRUCTURE

↓
Organizes the patient data
↓
ADT
↓
Defines required operations
↓
C IMPLEMENTATION
↓
Uses structures, pointers and functions
↓
EFFICIENT HEALTHCARE DATA PROCESSING

How It Improves System Performance

The hierarchy improves the smart healthcare system in several ways:

1. **Better organization** – Patient information is stored systematically.
2. **Faster access** – Appropriate data structures make searching and retrieving information easier.
3. **Efficient memory usage** – Pointers and dynamic structures can reduce unnecessary memory usage.
4. **Easy modification** – ADTs allow operations such as insertion, deletion, and updating.
5. **Scalability** – Dynamic structures can handle increasing numbers of patient records.
6. **Better processing** – Organized data allows healthcare information to be processed efficiently.
7. **Improved performance** – Efficient data organization reduces processing time and supports faster access to patient information.

Literature Integration

For the **20% Literature Integration** criterion, add a small section below your concept map using the **actual sources given in your textbook/course material**:

LITERATURE CONNECTION
↓
Studies on Data Structures
↓
Efficient organization of large datasets
↓
Studies on ADTs
↓
Clear separation of data and operations
↓
Studies on Healthcare Information Systems

↓

Faster and organized patient-data processing

↓

Improved healthcare system performance

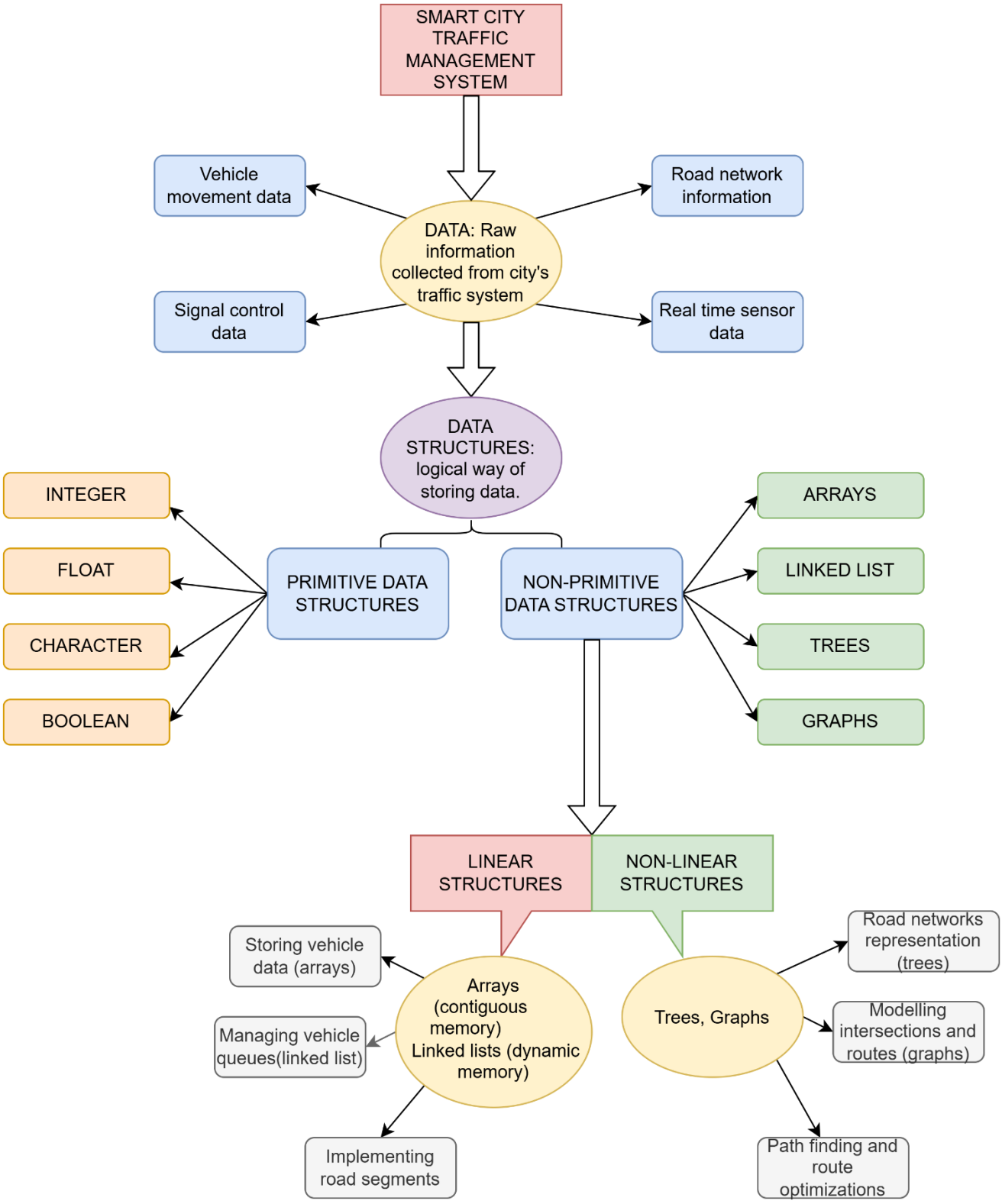
Important: If your faculty has provided specific papers, books, or references, write those actual authors/titles here rather than inventing references.

Final Conclusion

Conclusion

The concept map represents the hierarchical progression from Data to Data Object, Data Structure, Abstract Data Type, and finally C implementation. In a smart healthcare system, raw patient information is converted into meaningful data objects and organized using structures such as arrays, structures, and linked lists. Pointers provide efficient access and connection between data elements, while ADTs define the required operations such as insertion, deletion, searching, updating, and displaying. These concepts can be implemented in C using structures, pointers, arrays, and functions. This systematic approach improves data organization, memory utilization, processing efficiency, and accessibility, resulting in better overall performance of the smart healthcare system.

2)Concept Map



Explanation of the Concept Map

1. Data

Data refers to the raw traffic information collected by the **Smart City Traffic Management System**.

Examples include:

- Vehicle movement
- Vehicle count
- Vehicle speed
- Traffic density
- Road network information
- Traffic signal status

For example:

Vehicle count = 120 vehicles/minute

This raw information is the starting point of the traffic-management system.

---

2. Primitive Data Structures

Primitive data structures are the basic built-in data types used to represent simple traffic values.

Examples:

- **Integer (int)** – stores vehicle count.
- **Float (float)** – stores vehicle speed.
- **Character (char)** – stores signal state.
- **Boolean (boolean)** – stores signal status such as ON/OFF.

For example:

int vehicleCount = 120;

float speed = 45.5;

char signal = 'G';

These data types are simple, require less memory, and provide efficient storage of basic traffic information.

---

3. Non-Primitive Data Structures

Non-primitive data structures are used when traffic information becomes more complex and needs to be organized as a collection of related data.

They are mainly classified into:

- **Linear Data Structures**
- **Non-Linear Data Structures**

This classification depends on how the data elements are arranged and accessed.

---

4. Linear Data Structures

Linear data structures organize data elements in a sequential order, where elements are arranged one after another.

The main examples in the concept map are:

Array

An **array** stores multiple elements of the same type in a fixed-size sequential arrangement.

Traffic application:

An array can store vehicle counts from different lanes:

Lane 1 → 40 vehicles

Lane 2 → 35 vehicles

Lane 3 → 25 vehicles

Lane 4 → 20 vehicles

Arrays provide **fast indexing and direct access**, making them efficient when the required data position is known.

Linked List

A **linked list** stores data using connected nodes. Each node contains data and a link to another node.

Traffic application:

It can represent a dynamically changing **vehicle queue**, where vehicles enter and leave continuously.

Linked lists are useful when frequent **insertion and deletion** operations are required.

---

5. Non-Linear Data Structures

Non-linear data structures do not store data sequentially. They represent hierarchical or network relationships.

The main examples are:

- **Trees**
- **Graphs**

Trees

A **tree** represents data in a hierarchical structure consisting of nodes and branches.

Traffic application:

Trees can represent hierarchical road information, such as:

City

↓

Zone

↓

Road

↓

Intersection

↓

Lane

Trees are useful for **hierarchical searching and organization**.

Graphs

A **graph** consists of vertices and edges and is especially useful for representing connected networks.

Traffic application:

In a smart city:

- **Vertices** → intersections or locations
- **Edges** → roads connecting them

Graphs can therefore represent the complete **road network** and help in **route planning and traffic-flow analysis**.

---

6. Applications in Traffic Management

These data structures have different applications in a smart city.

Arrays

Used for:

- Vehicle queues
- Sensor readings
- Traffic counts
- Signal timing information

Linked Lists

Used for:

- Dynamically changing vehicle queues
- Vehicle entry and exit records
- Dynamic traffic records

Trees

Used for:

- Hierarchical road organization
- Searching road information
- Organizing intersections and zones

Graphs

Used for:

- Road-network representation
- Shortest-route selection
- Route planning
- Traffic-flow analysis
- Intersection connectivity

---

7. Functionality and Efficiency

Each data structure is selected according to its functionality and efficiency.

Data Structure	Main Function	Traffic Application
Array	Fast indexed access	Vehicle/sensor data
Linked List	Dynamic insertion and deletion	Vehicle queues
Tree	Hierarchical organization/search	Road hierarchy
Graph	Network relationship modelling	Road networks and routes

For example:

- **Arrays** provide fast direct access, approximately **O(1)** by index.

- **Linked lists** are efficient for insertion and deletion when the relevant position/node is known.
- **Trees** provide organized hierarchical searching.
- **Graphs** are suitable for representing complex road networks and finding routes.

---

8. Relationship Between the Concepts

The overall progression can be summarized as:

TRAFFIC DATA

↓

Raw vehicle and road information

↓

PRIMITIVE DATA STRUCTURES

↓

Basic values such as vehicle count, speed and signal status

↓

NON-PRIMITIVE DATA STRUCTURES

↓

Complex organization of traffic information

↓

LINEAR STRUCTURES

↓

Arrays + Linked Lists

↓

NON-LINEAR STRUCTURES

↓

Trees + Graphs

↓

FUNCTIONALITY & EFFICIENCY

↓

Selecting the appropriate structure according to the required operation

↓

SMART TRAFFIC MANAGEMENT

↓

Vehicle tracking + Route selection + Signal coordination + Congestion analysis

---

9. Why Are They Classified This Way?

The classification is based mainly on **how data is organized and how it can be accessed**.

Primitive

Primitive structures store **single, basic values**.

Example:

Vehicle speed = 45.5 km/h

Non-Primitive

Non-primitive structures store and manage **collections of related data**.

Linear

Linear structures arrange elements **sequentially**.

Example:

Vehicle 1 → Vehicle 2 → Vehicle 3 → Vehicle 4

Non-Linear

Non-linear structures represent **hierarchical or interconnected relationships**.

Example:

Intersection A ↔ Intersection B ↔ Intersection C

Therefore, the classification helps us select the right structure depending on the traffic-management requirement.

---

10. How It Improves Traffic Management

The use of appropriate data structures improves the smart city system in several ways:

1. **Better organization** – Traffic information is stored systematically.
2. **Faster access** – Arrays and other suitable structures allow quick retrieval of required data.
3. **Efficient updates** – Linked lists can handle dynamically changing vehicle information.
4. **Better route management** – Graphs can represent road connections and support route planning.
5. **Hierarchical organization** – Trees can organize roads, zones, and intersections.
6. **Efficient memory usage** – Choosing the appropriate structure avoids unnecessary storage.
7. **Improved traffic decisions** – Organized traffic data supports signal coordination and congestion analysis.
8. **Scalability** – The system can manage increasing numbers of vehicles, roads, and traffic records.

---

11. Literature Integration

For the **20% Literature Integration** criterion, add a small section below your concept map using the **actual sources provided by your faculty, textbook, or course material**.

LITERATURE CONNECTION

Studies on Data Structures

↓

Efficient organization and processing of large datasets

↓

Studies on Linear Data Structures

↓

Efficient sequential storage and dynamic data management

↓

Studies on Trees and Graphs

↓

Hierarchical and network-based data representation

↓

Studies on Intelligent Transportation Systems

↓

Traffic monitoring, route optimization and congestion management

↓

Smart City Traffic Management

↓

Efficient vehicle tracking + signal control + route planning

Important: Use your actual textbook papers/books or faculty-provided references here rather than creating fictional references.

---

12. Final Conclusion

Conclusion

The concept map represents the classification of data structures used in a **Smart City Traffic Management System**. Primitive data structures are used to store basic traffic information such as vehicle count, speed, and signal status. These are extended into non-primitive data structures, which are classified into **linear and non-linear structures**. Arrays and linked lists are useful for sequential and dynamically changing traffic data, while trees and graphs are suitable for hierarchical and network-based information. Each structure is selected according to its functionality, memory requirements, and efficiency. By applying these structures appropriately, the system can support **vehicle tracking, shortest-route selection, signal coordination, and congestion analysis**, resulting in efficient and organized smart-city traffic management.